

Two vertical yellow lines of varying heights on the left side of the slide.

2. Pruebas dinámicas de *software*

Introducción

Miguel Ángel Latre
Javier Nogueras Iso

Temario de la asignatura

1. El proceso de pruebas
-  2. Técnicas y herramientas de pruebas o *testing*
 - ❖ Introducción
 - ❖ Técnicas generales
 - ❖ Pruebas de unidad. Herramientas
 - ❖ Pruebas de integración. Técnicas y herramientas
 - ❖ Depuración
3. Técnicas y herramientas de evaluación estática
 - ❖ Revisiones
 - ❖ Análisis estático
 - Técnicas
 - Herramientas
4. Evaluación y *testing* de interfaces persona ordenador

Índice

- ❑ *Testing*
- ❑ Ejemplo
- ❑ Estrategia de pruebas
- ❑ Técnicas de diseño de pruebas
- ❑ Casos de prueba
- ❑ Ejecución de pruebas
- ❑ Herramientas
- ❑ Integración continua

Introducción

□ Pruebas o *testing*

- ❖ Ejecución total o parcial del código del sistema en un entorno controlado
- ❖ Objetivos
 - Evaluar la calidad del *software*
 - Evaluar los riesgos asociados a su despliegue
 - Encontrar defectos
 - Analizar el comportamiento/rendimiento del sistema en determinadas circunstancias
 - Refinar la especificación del código
- ❖ ¿Qué probar?
- ❖ ¿Cómo probarlo?

Introducción

❑ ¿Qué probar?

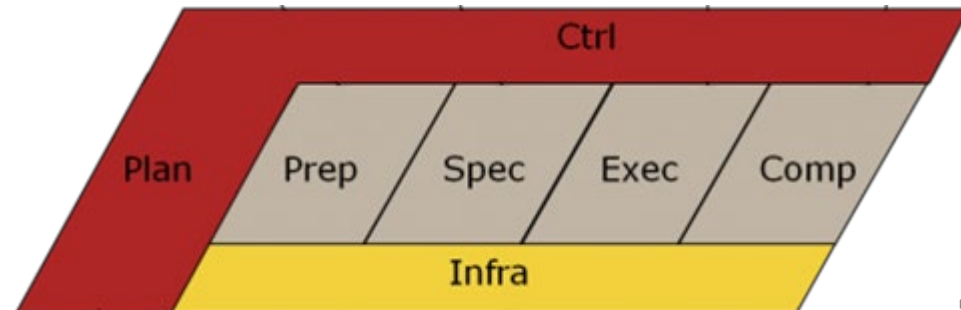
- ❖ No se puede probar todo.

- ❖ **Estrategia de pruebas:**

- Cuanto más importante sea algo, más habrá que probarlo.
- Los riesgos que se hayan identificado se **cubren** adecuadamente.

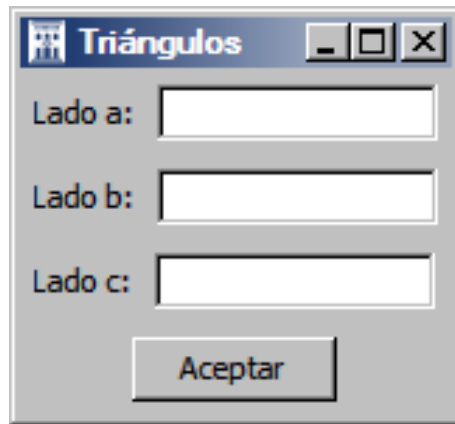
❑ ¿Cómo probarlo?

- ❖ Utilizando **técnicas de diseño de pruebas** para obtener **casos de prueba**
- ❖ **Ejecutando** esos casos de prueba
- ❖ **Analizando** los resultados de esos casos de prueba



Ejemplo

- ❑ Un programa lee tres valores **enteros** de un diálogo. Los tres valores representan las longitudes de los lados de un triángulo. El programa muestra un mensaje que indica si el triángulo es equilátero, isósceles o escaleno.
 - ❖ Myers, G. J.; Badgett, T.; Sandler, C. (2011). *The Art of Software Testing*, 3rd Edition. John Wiley and Sons Ltd.



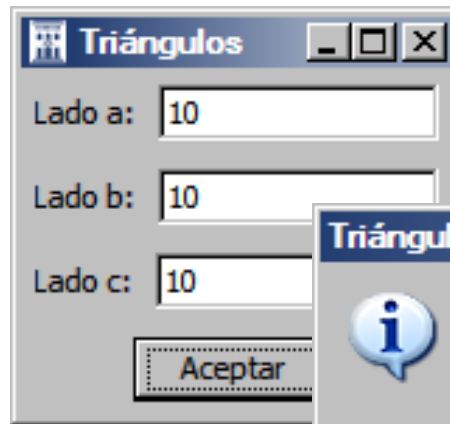
Triángulos

Lado a:

Lado b:

Lado c:

Aceptar



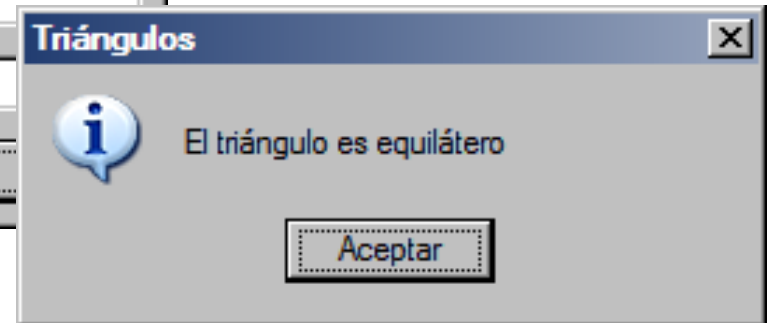
Triángulos

Lado a:

Lado b:

Lado c:

Aceptar



Triángulos

 El triángulo es equilátero

Aceptar

Ejemplo

- ❑ Un programa lee tres valores enteros de un diálogo. Los tres valores representan las longitudes de los lados de un triángulo. El programa muestra un mensaje que indica si el triángulo es equilátero, isósceles o escaleno.
 - ❖ Myers, G. J.; Badgett, T.; Sandler, C. (2011). *The Art of Software Testing*, 3rd Edition. John Wiley and Sons Ltd.
- ❑ Especificad casos de prueba para probar el programa de la forma más exhaustiva posible.
 - ❖ 5 minutos

Evaluación

- ❑ ¿Hay al menos un caso de prueba que representa un triángulo **válido**?
- ❑ ¿Hay al menos un caso de prueba que representa un triángulo **equilátero** válido?
- ❑ ¿Hay al menos un caso de prueba que representa un triángulo **isósceles** válido?
- ❑ ¿Hay al menos tres casos de prueba que representan triángulos isósceles válidos tales que se han probado las tres posibles **permutaciones** de los lados?
 - ❖ Ej.: (3, 3, 4), (3, 4, 3) y (4, 3, 3)
- ❑ ¿Hay al menos un caso de prueba en el que uno de los lados es **0**?

Evaluación

- ❑ ¿Hay al menos un caso de prueba en el que uno de los lados es **negativo**?
- ❑ ¿Hay al menos un caso de prueba en el que uno de los lados es **igual a la suma** de los otros dos?
- ❑ ¿Hay al menos tres casos de prueba en el que uno de los lados es igual a la suma de los otros, probando las tres posibles **permutaciones**?
- ❑ ¿Hay al menos un caso de prueba en el que los tres lados son mayores que 0 y el más largo es **mayor que la suma** de los otros dos?
- ❑ ¿Hay al menos tres casos de prueba como el anterior, probando todas las **permutaciones**?

Evaluación

- ❑ ¿Hay un caso de prueba con **todos** los lados igual a **0**?
- ❑ ¿Hay al menos un caso de prueba que utilice **valores no enteros**?
- ❑ ¿Hay al menos un caso de prueba en el que no se dé valor a alguno de los lados?
- ❑ ¿Para cada caso de prueba, has indicado cuál debe ser el **comportamiento esperado** del programa para ese caso?
- ❑ ¿Hay al menos un caso de prueba que sea un **triángulo escaleno no rectángulo**?

Evaluación

- Programadores experimentados obtienen de media 7,8 sobre 14 [Myers]

Estrategia de las pruebas

- A partir del análisis de riesgo de los productos
 - ❖ Qué características o partes del sistema deben ser probados
 - ❖ Con qué nivel de detalle
 - Pruebas dinámicas (*testing*) exhaustivas, medias, limitadas; pruebas estáticas; pruebas implícitas
 - ❖ En qué nivel de pruebas
 - Unidad, integración, sistema, aceptación

Técnicas de diseño de pruebas

□ Repaso de Ingeniería del *Software*:

¿Qué técnicas de diseño de pruebas habrían producido casos de prueba similares a los del ejemplo de los triángulos?

❖ Técnicas basadas en la especificación (caja negra):

➤ Particiones de equivalencia

- Los tres parámetros y el tipo de resultado

➤ Análisis de valores límite

➤ Valores típicos de error y valores imposibles

- (aplicado en el contexto de la teoría de triángulos)

Técnicas de diseño de pruebas

❑ **Definición:** Métodos definidos para obtener, a partir de unos requisitos o especificaciones determinados, casos de prueba que proporcionan una determinada cobertura para los mismos

❑ **Ventajas**

- ❖ Más efectivo que diseñar pruebas *ad hoc*
- ❖ Para cada técnica, se conoce el tipo y grado de cobertura. Se adapta a la exigencia de la estrategia de pruebas elegida.
- ❖ El diseño de la prueba es repetible:
 - La definición del método permite independizar de la persona o equipo que realiza la prueba.
 - La definición del método permite que las pruebas sean transferibles y mantenibles.
- ❖ Facilitan la gestión de las pruebas.

Casos de prueba

❑ Situación de prueba

- ❖ Condición aislada y particular bajo la que el objeto de prueba muestra un comportamiento específico que quiere probarse

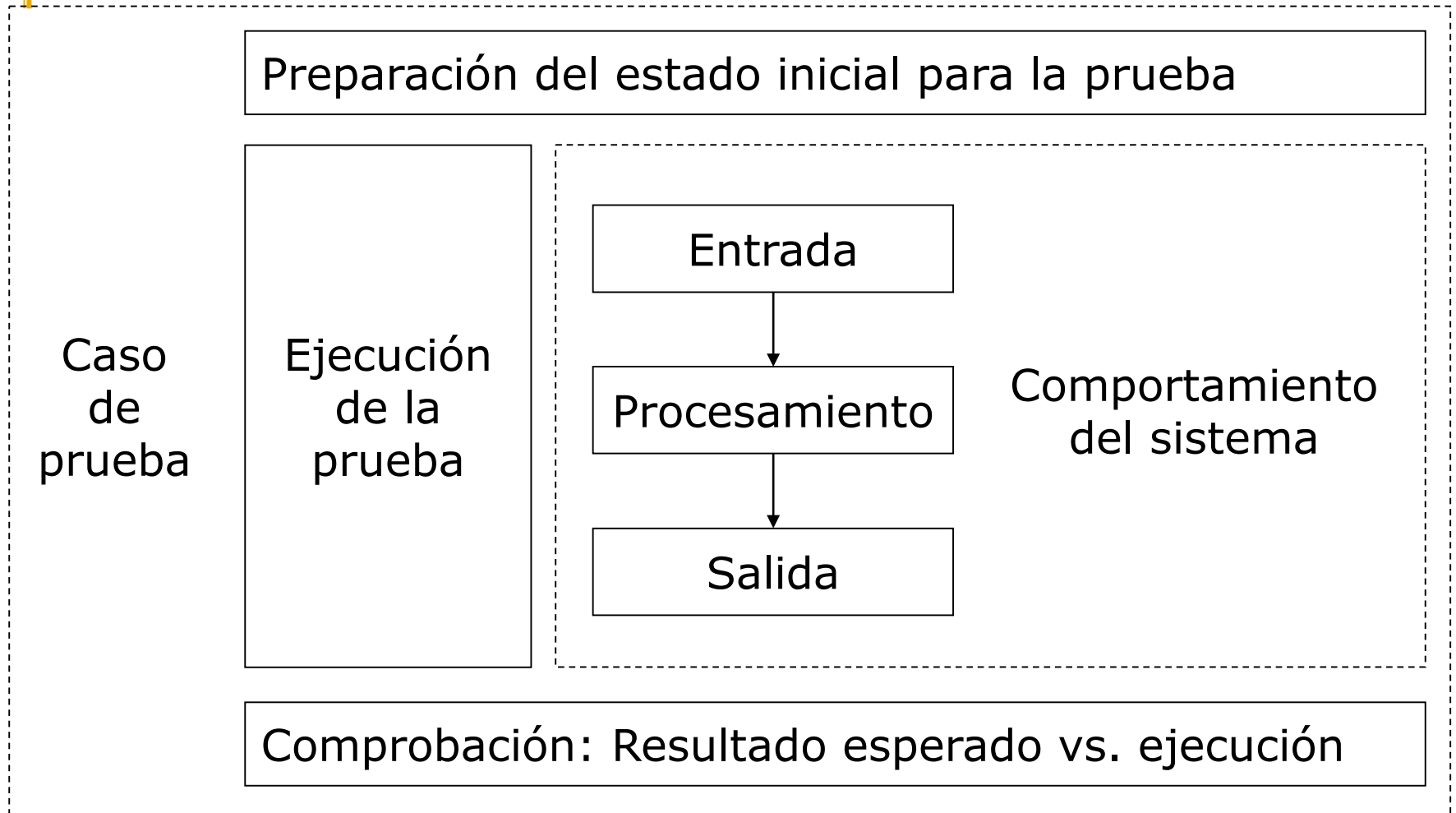
❑ Caso de prueba

- ❖ Caso concreto que se utiliza para determinar si el objeto de prueba muestra el comportamiento específico que se desea
- ❖ Representa un todo ejecutable, que combina una o varias situaciones de prueba.

❑ Prueba

- ❖ Ejecución controlada de un caso de prueba o un conjunto de casos de prueba

Casos de prueba



Ejecución de pruebas

□ Dependiente del nivel de la prueba

- ❖ De sistema o de aceptación
- ❖ De integración
- ❖ De unidad

□ Características imprescindibles

- ❖ Validez

□ Características deseables

- ❖ Automatización
 - Necesidad de herramientas
- ❖ Ejecución rápida

Ejecución de pruebas de unidad

```
package es.unizar.eina.vv.testing.triangles;

/**
 * Enum class that represents the different types a
 * triangle can be classified as (equilateral,
 * isosceles, and scalene) based on the length of its
 * sides.
 */
public enum TriangleType {
    EQUILATERAL, ISOSCELES, SCALENE;
}
```

Ejecución de pruebas de unidad

```
public class Triangle {  
    /**  
     * Constructs a triangle whose sides measure a, b and c units.  
     *  
     * @param a  
     * the integer length of the first side of the triangle.  
     * @param b  
     * the integer length of the second side of the triangle.  
     * @param c  
     * the integer length of the third side of the triangle.  
     * @throws IllegalArgumentException  
     * when any of the sides is less than or equal to zero or  
     * when their lengths do not allow forming a triangle  
     * because the sum of the lengths of the two smaller sides  
     * is less than or equal to the length of the longest side.  
     */  
    public Triangle(int a, int b, int c) throws IllegalArgumentException;  
    ...  
}
```

Ejecución de pruebas de unidad

...

*/***

** Returns the type of this triangle (equilateral, isosceles or
* scalene)*

** @return the type of this triangle.*

**/*

public TriangleType type();

}

Ejecución de pruebas de unidad

```
/**
 * Program that checks that Triangle.type() works correctly
 */
public static void main(String[] args) {
    Triangle triangle = new Triangle(3, 4, 5);
    if (triangle.type() == TriangleType.SCALENE) {
        System.out.println("new Triangle(3, 4, 5).type() OK");
    } else {
        System.out.println("Error en new Triangle(3, 4, 5).type()");
    }

    triangle = new Triangle(3, 3, 3);
    if (triangle.type() == TriangleType.EQUILATERAL) {
        System.out.println("new Triangle(3, 3, 3).type() OK");
    } else {
        System.out.println("Error en new Triangle(3, 3, 3).type()");
    }
}
```

JUnit

- ❑ Conjunto de bibliotecas para hacer pruebas de unidad de aplicaciones Java
- ❑ Automatiza la ejecución de pruebas sobre clases y métodos Java de manera controlada, para poder evaluar si el funcionamiento de cada uno de los métodos de la clase se comporta como se espera.
- ❑ Permite realizar pruebas de regresión.
- ❑ Integrado en IntelliJ, Android Studio y otros entornos

JUnit 4



JUnit

```
package es.unizar.eina.vv.testing.triangles;

import org.junit.jupiter.api.Test;
import static
    es.unizar.eina.vv.testing.triangles.TriangleType.*;
import static org.junit.jupiter.api.Assertions.*;

class TriangleTest {
    @Test
    void testTriangleSides_3_3_3_isEquilateral() {
        Triangle triangle = new Triangle(3, 3, 3);
        TriangleType type = triangle.type();
        assertEquals(EQUILATERAL, type);
    }
    ...
}
```

Ejecución de pruebas de unidad

The screenshot shows an IDE with a project named 'unizar-vv-triángulos-solucion'. The project structure includes a 'test' directory with a 'java' subdirectory containing 'TrianguloTest'. The code for 'TrianguloTest.java' is displayed, showing three test methods: 'testTrianguloLados_3_3_3_esEquilatero()', 'test_tipo_p01_equilatero()', and 'test_tipo_p02_isosceles_ladoC()'. The 'test_tipo_p01_equilatero()' method is highlighted, showing a failure. The error message indicates that the expected result is 'equilátero' but the actual result is 'isósceles'.

```
package es.unizar.eina.vv6f.pruebas.triángulos;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class TrianguloTest {

    @Test
    void testTrianguloLados_3_3_3_esEquilatero() {
        Triangulo triangulo = new Triangulo( a: 3, b: 3, c: 3);
        TipoTriangulo tipoObtenido = triangulo.tipo();
        assertEquals(TipoTriangulo.EQUILATERO, tipoObtenido);
    }

    @Test
    void test_tipo_p01_equilatero() {
        Triangulo triangulo = new Triangulo( a: 3, b: 2, c: 3);
        assertEquals(TipoTriangulo.EQUILATERO, triangulo.tipo());
    }

    @Test
    void test_tipo_p02_isosceles_ladoC() {
        Triangulo triangulo = new Triangulo( a: 3, b: 3, c: 2);
    }
}
```

Run: TrianguloTest (es.unizar.e 15 ms)

- test_tipo_p07_negativo 11 ms
- test_tipo_p15_plano_ladoC() 2 ms
- test_tipo_p02_isosceles 2 ms
- testTrianguloLados_3_3_3_esEquilatero 2 ms
- test_tipo_p09_equilatero_limite() 2 ms
- test_tipo_p14_plano_ladoB() 2 ms
- test_tipo_p06_negativo_ladoA() 2 ms
- test_tipo_p13_plano_ladoA() 2 ms
- test_tipo_p01_equilatero 1 ms
- test_tipo_p22_isosceles_ladoB_m 2 ms

Tests failed: 1, passed: 23 of 24 tests - 15 ms

org.opentest4j.AssertionFailedError:
Expected :equilátero
Actual :isósceles
<Click to see difference>

<5 internal lines>

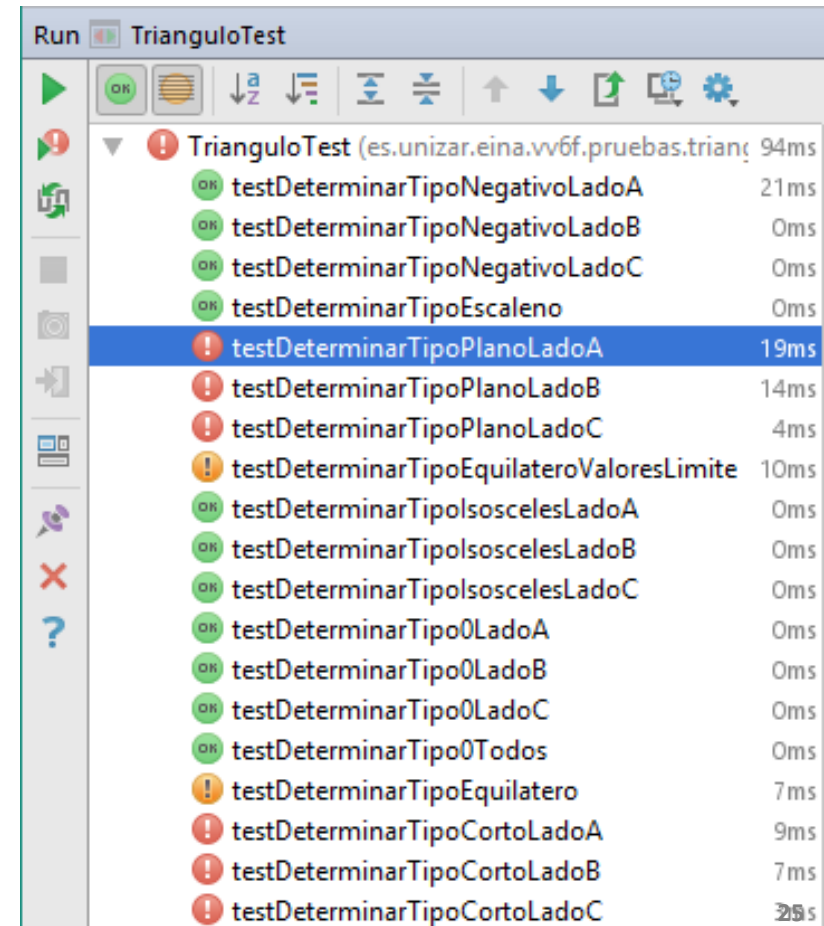
at es.unizar.eina.vv6f.pruebas.triángulos.TrianguloTest.test_tipo_p01_equilatero(TrianguloTest.java:18) <29 internal lines>
at java.base/java.util.ArrayList.forEach(ArrayList.java:1596) <9 internal lines>
at java.base/java.util.ArrayList.forEach(ArrayList.java:1596) <27 internal lines>

unizar-vv-triángulos-solucion > src > test > java > es > unizar > eina > vv6f > pruebas > triángulos > TrianguloTest > test_tipo_p01_equilatero 17:49 CRLF UTF-8 4 spaces

Análisis con JUnit

Distinción entre

- ❖ Fallos (*failures*) 
 - Aserciones que no se verifican
- ❖ Errores (*errors*) 
 - Excepciones no esperadas



Run	TrianguloTest
OK	testDeterminarTipoNegativoLadoA 21ms
OK	testDeterminarTipoNegativoLadoB 0ms
OK	testDeterminarTipoNegativoLadoC 0ms
OK	testDeterminarTipoEscaleno 0ms
!	testDeterminarTipoPlanoLadoA 19ms
!	testDeterminarTipoPlanoLadoB 14ms
!	testDeterminarTipoPlanoLadoC 4ms
!	testDeterminarTipoEquilateroValoresLimite 10ms
OK	testDeterminarTipososcelesLadoA 0ms
OK	testDeterminarTipososcelesLadoB 0ms
OK	testDeterminarTipososcelesLadoC 0ms
OK	testDeterminarTipo0LadoA 0ms
OK	testDeterminarTipo0LadoB 0ms
OK	testDeterminarTipo0LadoC 0ms
OK	testDeterminarTipo0Todos 0ms
!	testDeterminarTipoEquilatero 7ms
!	testDeterminarTipoCortoLadoA 9ms
!	testDeterminarTipoCortoLadoB 7ms
!	testDeterminarTipoCortoLadoC 25s

Ejecución de pruebas de unidad

- ❑ El código a probar puede depender de otros elementos que pueden no estar disponibles
 - ❖ Obtienen información del exterior del sistema
 - ❖ Su estado es difícil de crear o reproducir
 - ❖ Es lento
 - ❖ Todavía no existe
 - ❖ Debería incluir atributos o métodos exclusivamente para las pruebas
- ❑ Ejemplos
 - ❖ Sensor de temperatura actual
 - ❖ Errores de conexión
 - ❖ Bases de datos, servidores web
- ❑ **Objetos simulados** (*Mock objects*)
 - ❖ *Mock, fake, stub*

Ejecución de pruebas de integración

- ❑ En algunos casos, pueden usarse las mismas herramientas que para pruebas de unidad.
 - ❖ Para utilizarlas, se pueden utilizar técnicas que mejoran la calidad «estática» de código.
 - Ejemplo: en Android, los objetos View deberían ser solo interfaz, y la lógica ir en una clase Java que se puede probar con JUnit
- ❑ Su ejecución puede ser más lenta.
- ❑ En entornos de **integración continua**, las pruebas de integración no las lanza el desarrollador, sino un sistema centralizado.

Integración continua (*continuous integration*)

- ❑ Propuesto inicialmente por Grady Booch y defendido por Martin Fowler.
- ❑ Práctica consistente en realizar integraciones automáticas de un proyecto lo más a menudo posible
 - ❖ Compilación y ejecución de *test* automáticos de todo un proyecto
 - ❖ Requiere de un gestor de versiones.
 - ❖ Periódicamente o por eventos, se descarga el código del gestor de versiones, se compila, ejecutan las pruebas y se emite informes.
 - ❖ Detección temprana de errores y fallos
- ❑ Entornos específicos
 - ❖ Jenkins, Travis CI, ...
 - ❖ GitHub lista 8623 herramientas de integración continua que se integran con GitHub (<https://github.com/marketplace?query=continuous+integration>)
- ❑ A menudo la integración continua está asociada con las metodologías de **programación extrema** y **desarrollo ágil**.
- ❑ https://en.wikipedia.org/wiki/Continuous_integration

Ejecución de pruebas de sistema o aceptación

- ❑ Como un caso de pruebas de integración
- ❑ Interfaces gráficas de usuarios
 - ❖ *Ad hoc*
 - ❖ Herramientas específicas
 - Selenium para aplicaciones web
 - Robotium, Espresso, UI Automator para Android
 - AutoHotkey para Windows
 - Maveryx para Java
 - Sikuli para web o escritorio
 - ...



Problema introductorio

- Extensión de la técnica de particiones de equivalencia
 - ❖ Inclusión de los datos de salida
 - ❖ Particiones de equivalencia dependientes de más de un parámetro
- Herramientas
 - ❖ JUnit
 - ❖ Medición de cobertura

Ejecución de pruebas de unidad

Triangle

...

```
+ <<Create>> Triangle(a, b, c: int) {exception IllegalArgumentException}  
+ type(): TriangleType
```

«enumeration»
TriangleType

EQUILATERAL
ISOSCELES
SCALENE

Ejecución de pruebas de unidad

```
package es.unizar.eina.vv.testing.triangles;

/**
 * Enum class that represents the different types a
 * triangle can be classified as (equilateral,
 * isosceles, and scalene) based on the length of its
 * sides.
 */
public enum TriangleType {
    EQUILATERAL, ISOSCELES, SCALENE;
}
```

Ejecución de pruebas de unidad

```
public class Triangle {  
    /**  
     * Constructs a triangle whose sides measure a, b and c units.  
     *  
     * @param a  
     * the integer length of the first side of the triangle.  
     * @param b  
     * the integer length of the second side of the triangle.  
     * @param c  
     * the integer length of the third side of the triangle.  
     * @throws IllegalArgumentException  
     * when any of the sides is less than or equal to zero or  
     * when their lengths do not allow forming a triangle  
     * because the sum of the lengths of the two smaller sides  
     * is less than or equal to the length of the longest side.  
     */  
    public Triangle(int a, int b, int c) throws IllegalArgumentException;  
    ...  
}
```

Ejecución de pruebas de unidad

...

*/***

** Returns the type of this triangle (equilateral, isosceles or
* scalene)*

** @return the type of this triangle.*

**/*

public TriangleType type();

}

Problema

❑ Particiones de equivalencia

- ❖ Parámetros a, b, c del constructor
- ❖ Extensión de la técnica de particiones de equivalencia
 - Inclusión del resultado del método tipo()
 - Inclusión de valores límite
 - Consideración de dependencias entre parámetros (procedentes del dominio del problema)

❑ Escritura de *tests* con JUnit

❑ Implementación del código

❑ Análisis de la cobertura

Particiones de equivalencia

Parámetro	Particiones de equivalencia válidas		Particiones de equivalencia no válidas	
a	1	$a > 0$	4	$a \leq 0$
b	2	$b > 0$	5	$b \leq 0$
c	3	$c > 0$	6	$c \leq 0$

Particiones de equivalencia considerando también el resultado

Parámetro	Particiones de equivalencia válidas		Particiones de equivalencia no válidas	
a	1	$a > 0$	7	$a \leq 0$
b	2	$b > 0$	8	$b \leq 0$
c	3	$c > 0$	9	$c \leq 0$
<i>resultado</i>	4	EQUILATERAL		
	5	ISOSCELES		
	6	SCALENE		

Casos de prueba

a	b	c	Resultado	Clases cubiertas
3	3	3	EQUILATERAL	1,2,3,4
3	3	2	ISOSCELES	1,2,3,5
3	4	5	SCALENE	1,2,3,6
-2	3	3	IllegalArgumentException	7
3	-2	3	IllegalArgumentException	8
3	3	-2	IllegalArgumentException	9

Nota: en los tres primeros casos, el resultado corresponde a la creación del triángulo y la invocación al método `tipo()`, mientras que en los tres últimos, corresponde solo a la creación del triángulo.

Casos de prueba añadiendo valores límite

a	b	c	resultado	clases cubiertas
3	3	3	EQUILATERAL	1,2,3,4
3	3	2	ISOSCELES	1,2,3,5
3	4	5	SCALENE	1,2,3,6
-2	3	3	IllegalArgumentException	7
3	-2	3	IllegalArgumentException	8
3	3	-2	IllegalArgumentException	9
1	1	1	EQUILATERAL	valores límite clases 1, 2, 3
0	3	3	IllegalArgumentException	valor límite clase 7
3	0	3	IllegalArgumentException	valor límite clase 8
3	3	0	IllegalArgumentException	valor límite clase 9

Casos de prueba

□ Casos de prueba exhaustivos

- ❖ Particiones de equivalencia
- ❖ Valores límite
- ❖ Dominio del problema [Mayers, 2011]
 - Permutaciones
 - Teoría de triángulos
- ❖ Técnicas basadas en la experiencia

Prueba	a	b	c	Resultado	Clases cubiertas. Comentarios
P1	3	3	3	EQUILATERAL	1,2,3,4
P2	3	3	2	ISOSCELES	1,2,3,5
P3	3	2	3	ISOSCELES	1,2,3,5 (permutación)
P4	2	3	3	ISOSCELES	1,2,3,5 (permutación)
P5	3	4	5	SCALENE	1,2,3,6
P6	-2	3	3	IllegalArgumentException	8
P7	3	-2	3	IllegalArgumentException	9
P8	3	3	-2	IllegalArgumentException	10
P9	1	1	1	EQUILATERAL	valores límite clases 1, 2, 3
P10	0	3	3	IllegalArgumentException	valor límite clase 8
P11	3	0	3	IllegalArgumentException	valor límite clase 9
P12	3	3	0	IllegalArgumentException	valor límite clase 10
P13	1	2	7	IllegalArgumentException	teoría triángulos
P14	1	7	2	IllegalArgumentException	—, permutación
P15	7	1	2	IllegalArgumentException	—, —
P16	3	4	7	IllegalArgumentException	—, valores límite
P17	3	7	4	IllegalArgumentException	—, —, permutación
P18	7	3	4	IllegalArgumentException	—, —, —
P19	0	0	0	IllegalArgumentException	[Mayers, 2011]
P20	3	4	6	SCALENE	escaleno no rectángulo

JUnit

```
package es.unizar.eina.vv.testing.triangles;

import org.junit.jupiter.api.Test;
import static
    es.unizar.eina.vv.testing.triangles.TriangleType.*;
import static org.junit.jupiter.api.Assertions.*;

class TriangleTest {
    @Test
    void testTriangleSides_3_3_3_isEquilateral() {
        Triangle triangle = new Triangle(3, 3, 3);
        TriangleType type = triangle.type();
        assertEquals(EQUILATERAL, type);
    }
    ...
}
```

JUnit

- ❑ Un test en JUnit es un método público de una clase, etiquetado la etiqueta `@Test`.
- ❑ El nombre conviene que sea muy descriptivo del objetivo del test, incluso si queda largo, como en el del ejemplo anterior.
- ❑ Un test suele componerse de tres partes:
 - ❖ Preparación *(Given)*
 - ❖ Ejecución *(When)*
 - ❖ Verificación *(Then)*

JUnit

□ En el ejemplo anterior:

❖ Preparación

- La primera instrucción, en la que se crea un nuevo objeto de la clase `Triangle`.

❖ Ejecución

- La expresión en la que se invoca al método `type()`.

❖ Verificación

- Invocación al método `assertEquals`, donde se comprueba si el resultado de la invocación al método `type()` es igual al valor `EQUILATERAL`.
- El convenio en JUnit es poner en primer lugar el resultado esperado por el test y en segundo lugar el resultado obtenido al ejecutar el objeto bajo prueba.

JUnit

```
package es.unizar.eina.vv.testing.triangles;

import org.junit.jupiter.api.Test;
import static
    es.unizar.eina.vv.testing.triangles.TriangleType.*;
import static org.junit.jupiter.api.Assertions.*;

class TriangleTest {
    @Test
    void testTriangleSides_3_3_3_isEquilateral() {
        Triangle triangle = new Triangle(3, 3, 3);
        assertEquals(EQUILATERAL, triangle.type());
    }
    ...
}
```

JUnit

```
package es.unizar.eina.vv6f.pruebas.triángulos;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class TriangleTest {

    @Test
    public void testFirstSideIsPositive () {
        assertThrows(IllegalArgumentException.class,
            () -> new Triangle(-2, 3, 3));
    }

    ...
}
```

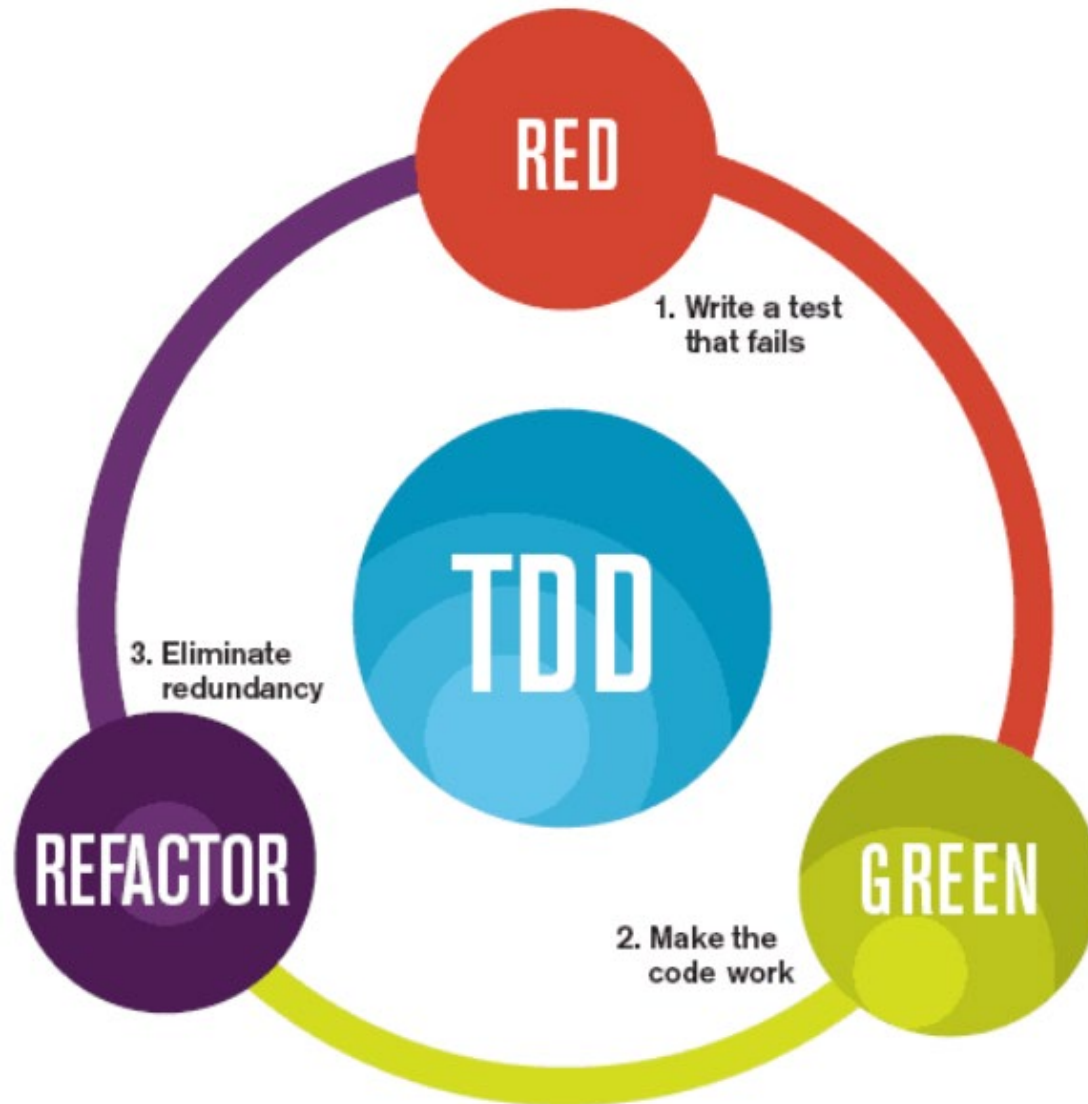
Ejecución con medición de cobertura

□ En IntelliJ (y AndroidStudio)

❖ Modos de ejecución desde el entorno

- Ejecución
- Depuración
- Ejecución con medición de cobertura

Test-driven development (TDD)



The mantra of Test-Driven Development (TDD) is “red, green, refactor.”

Repositorio unizar-vv-triangles

□ <https://github.com/miguel-latre/unizar-vv-triangles>

❖ Instrucciones del ejercicio en el fichero README.md

